Types primitifs

Un entier se note **`int`**, un entier long **`long`**, un réel **`float`** ou **`double`** et un booléen se note **`boolean`** ayant pour valeur **`true`** ou **`false`**. Une chaîne de caractères est représentée par une instance de la classe **`String`**... Il ne s'agit donc pas d'un type primitif.

Commentaires

```
// ceci est un commentaire sur une seule ligne
/* cela est
   un commentaire sur
   plusieurs lignes*/
```

Classes et attributs

Une classe se définit comme suit :

```
class NomClasse {
     // Attributs
     TypeAttribut nomAttribut;
     …
     // Méthodes de la classe
     …
}
```

Pour définir l'héritage d'une classe parent C par une classe SC, on définit SC de la façon suivante :

```
class SC extends C {
     // définition de la classe…
}
```

Une classe dont le nom est `NomClasse` doit obligatoirement être saisie dans un fichier de nom `NomClasse.java`. Pour compiler un tel fichier source Java, appeler le compilateur de la façon suivante :

```
javac NomClasse.java
```

Une fois la compilation exécutée, s'il n'y a pas d'erreur (auquel cas, les corriger), le compilateur a créé un fichier `NomClasse.class` qui contient la version compilée de la classe.

Méthodes

Les méthodes sont déclarées et définies à l'intérieur de la classe. La syntaxe de Java est très largement inspirée de celle du C :

- le type de retour d'une méthode est déclaré juste avant son nom ;
- pour définir une méthode qui ne retourne pas de résultat, le type de retour est **`void`** ;
- les paramètres sont signalés par leur type puis leur nom, et séparés d'une virgule ;
- l'affectation se fait par = et le test d'égalité par ==, les syntaxes de **`if`**, **`switch`**, **`while`** et **`for`** sont identiques.

```
class Exemple {
     void methode1() {
          System.out.println("essai");
     }
     int somme(int a, int b) {
          return a + b;
     }
     boolean positifP(int a) {
          if (a > 0)
               return true;
          else
               return false;
     }
}
```

Instanciation

Pour instancier une classe, c'est à dire créer un objet de cette classe, on doit déclarer une référence puis créer l'instance en utilisant **`new`** suivi d'un constructeur de la classe à instancier. Notez que la déclaration et la création de l'objet peut se faire sur une seule ligne.

```
NomClasse o;         // déclaration[1]
o = new NomClasse(); // création de l'objet
AutreClasse o2 = new AutreClasse(...);
```

Il est évidemment possible d'utiliser une référence comme type d'un attribut d'une classe, comme type d'un paramètre de méthode ou comme type de retour d'une méthode.

```
class Exemple {
     AutreClasse objetMemorise;
     AutreClasse obtenirValeur() {
          return objetMemorise;
     }
     void fixerValeur(AutreClasse i) {
          objetMemorise = i;
     }
}
```

Référence `null`

`null` est une valeur de référence particulière (non typée, pouvant donc être utilisée avec tout type de référence) qui ne repère aucun objet. Quand une référence est créée, elle est implicitement initialisée à **`null`**, sauf indication contraire. Il est évidemment interdit d'accéder à des attributs ou d'appeler des méthodes à partir d'une référence **`null`**. Cela générera un avertissement à la compilation.

Constructeurs et appel du constructeur de la super-classe

Lorsqu'on définit une classe `C`, on peut définir des constructeurs avec ou sans paramètres pour initialiser les objets créés. Tout constructeur d'une classe autre que celui de la classe **`Object`** (voir plus loin) fait appel soit à un constructeur de la super-classe, soit à un constructeur de la même classe. Cet appel doit être la première instruction dans la définition du constructeur. On utilise la syntaxe **`super`**`(arguments)` (où `arguments` peut être vide ou non) pour appeler un constructeur de la superclasse, et **`this`**`(arguments)` pour appeler un constructeur de la même classe. Attention, ces intructions doivent être les premières du constructeur.

Si aucun constructeur de la super-classe ou de la même classe n'est appelé explicitement, le compilateur génère implicitement un appel au constructeur par défaut **`super`**`()` qui est sans paramètre (il faut qu'il y ait un tel constructeur dans la super-classe sinon une erreur est détectée).

```
class Sc extends C {
     public Sc(String s) {
          // appel implicite au constructeur C()
          … // initialisation propre à Sc (gestion de s par exemple)
     }
     public Sc(int a, String s){
          super(a); // appel du constructeur C(int)
          … // initialisation propre à Sc (gestion de s par exemple)
     }
}
```

Appel de la super méthode

Quand une méthode est redéfinie dans une sous-classe, il est souvent nécessaire d'appeler la méthode masquée (méthode de même signature de la super-classe). Cet appel peut être fait n'importe où dans le code de le méthode de la sous-classe. L'appel se fait en utilisant la pseudo-variable **`super`** qui, à la manière de **`this`**, repère l'objet sur lequel la méthode est en train d'être exécutée, mais qui est du type de la super-classe. Ainsi, elle n'est pas nécessairement la première instruction de la méthode.

```
class Sc extends C {
     public void methode(int a) { /* redéfinition d'une méthode
                                     définie dans C */
          …
          super.methode(a);
          …
     }
}
```

La classe `Object` : racine de l'arbre d'héritage

Si une classe est déclarée comme n'ayant aucune super-classe, elle admet implicitement **`Object`** comme super-classe. La classe **`Object`** fait partie du paquetage `java.lang`, qui contient les classes et les interfaces les plus centrales du langage.

[1]Quand une référence est déclarée, elle est (implicitement) initialisée à la valeur `null`.

Getters / Setters

Afin de rendre les classes plus robustes, on peut interdire la modification libre pour vérifier que les champs correspondent bien à certains critères. Pour cela, on déclare les champs privés grâce à l'attribut **`private`** qui le rend inaccessible en dehors de la classe. Pour pouvoir accéder à ou modifier sa valeur, on utilise deux méthodes pour chacun des champs : un getter qui permet d'accéder à sa valeur et un setter qui permet de modifier sa valeur.

```
class C {
     private int champ1;
     private String champ2;

     public int getChamp1() {
          return this.champ1;
     }

     public void setChamp1(int champ1) {
          this.champ1 = champ1;
     }
}
```

Écrire dans la console

Pour écrire dans la console, on utilise les méthodes **`System.`***`out`***`.print()`** et **`System.`***`out`***`.println()`**. Ces méthodes sont identiques, si ce n'est que **`System.`***`out`***`.println()`** passe à la ligne après l'écrire dans la console. Ces méthodes prennent n'importe quel type en paramètre.

Pour écrire des données de types différents lors du même appel à **`System.`***`out`***`.print()`** ou **`System.`***`out`***`.println()`**, on construit une chaîne de caractères contenant toutes les données en les concaténant avec **+** :

```
class C {
     …
     public void afficher() {
          System.out.println("champ1: " + champ1);
          System.out.print("champ2: ")
          System.out.print(champ2);
          System.out.println();      // passe simplement à la ligne
     }
}
```

Programme principal

Pour écrire un programme de test, il est nécessaire d'écrire une méthode particulière dans une nouvelle classe ou une des classes du projet. Cette méthode devra avoir la signature suivante :

```
class NomClasse {
     public static void main(String[] args) {
          …
     }
}
```

Une classe possédant une telle méthode `main` peut être « exécutée ». C'est à dire qu'il est possible de lancer l'exécution de cette méthode `main` en appelant la machine virtuelle Java de la façon suivante (dans le répertoire contenant le fichier `NomClasse.class` obtenu après compilation) :

```
java NomClasse
```